Jhanvi Cars – AWS Cross-Region DR Database Runbook

PostgreSQL on Amazon RDS • Cross-Region Replication • Validation • Failover • Failback

Reference lab: Primary region ap-south-2, DR region ap-south-1, PostgreSQL/RDS, EKS, AWS ALB, Ingress and Route 53. Adjust names, regions and commands to your actual environment.

1. DR Objectives

RPO (Recovery Point Objective) = maximum acceptable data loss. Example: RPO 5 minutes means the DR design should normally keep the recoverable data loss within approximately five minutes.
RTO (Recovery Time Objective) = maximum acceptable recovery time. Example: RTO 30 minutes means service should be restored within 30 minutes of the disaster.

2. Target Architecture

```
NORMAL OPERATION

Users
  |
  v
Route 53
  |
  v
Primary ALB
  |
  v
Primary EKS / Ingress
  |
  v
Primary RDS PostgreSQL
  |
  | cross-region replication
  v
DR RDS PostgreSQL

DISASTER

Primary Region X
       |
       v
Validate replication -> Promote DR DB
       |
       v
DR EKS + ALB
       |
       v
Route 53 -> DR ALB
       |
       v
Users
```

3. Phase A – Inventory the Primary Database

Check RDS status and configuration:

```
aws rds describe-db-instances \
  --region ap-south-2 \
  --db-instance-identifier jhanvi-cars-prod

aws rds describe-db-instances \
  --region ap-south-2 \
  --db-instance-identifier jhanvi-cars-prod \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    Engine:Engine,
```

```
    Version:EngineVersion,
    Class:DBInstanceClass,
    Storage:AllocatedStorage,
    Endpoint:Endpoint.Address,
    Port:Endpoint.Port
  }'
```

Connect with psql:

```
psql \
  -h <primary-rds-endpoint> \
  -p 5432 \
  -U <username> \
  -d <database>
```

Baseline checks:

```
SELECT version();
SELECT current_database();
SELECT now();

SELECT datname
FROM pg_database
WHERE datistemplate = false;

SELECT
    schemaname,
    tablename
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY schemaname, tablename;

SELECT extname, extversion
FROM pg_extension
ORDER BY extname;

SELECT
    rolname,
    rolsuper,
    rolcreaterole,
    rolcreatedb
FROM pg_roles
ORDER BY rolname;

SELECT pg_size_pretty(pg_database_size(current_database()));
```

Record critical business-table counts:

```
SELECT count(*) FROM customers;
SELECT count(*) FROM drivers;
SELECT count(*) FROM vehicles;
SELECT count(*) FROM bookings;
```

4. Phase B – Prepare the DR Region

Prepare the DR VPC, private subnets, route tables, security groups, DB subnet group, KMS configuration where applicable, and connectivity from DR EKS to DR RDS. Keep database port 5432 private; allow it from the DR application security group rather than from the public Internet.

```
aws rds describe-db-subnet-groups \
  --region ap-south-1

aws ec2 describe-security-groups \
  --region ap-south-1
```

5. Phase C – Establish Cross-Region Replication

For a supported RDS PostgreSQL configuration, create a cross-Region read replica using the AWS RDS API/CLI. Exact parameters depend on encryption, KMS keys, subnet groups, instance class and other settings.

```
aws rds create-db-instance-read-replica \
  --db-instance-identifier jhanvi-cars-dr \
  --source-db-instance-identifier <primary-db-arn> \
  --region ap-south-1
```

Monitor creation:

```
aws rds describe-db-instances \
  --region ap-south-1 \
  --db-instance-identifier jhanvi-cars-dr \
  --query 'DBInstances[0].DBInstanceStatus'
```

6. Phase D – Validate Replication Health

Do not treat an AWS status of 'available' as proof that DR is ready. Validate replication status, lag, errors, storage, database consistency and application connectivity.

6.1 Check replica relationship

```
aws rds describe-db-instances \
  --region ap-south-1 \
  --db-instance-identifier jhanvi-cars-dr \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    Source:ReadReplicaSourceDBInstanceIdentifier,
    Endpoint:Endpoint.Address
  }'
```

6.2 Measure replication lag

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReplicaLag \
  --dimensions Name=DBInstanceIdentifier,Value=jhanvi-cars-dr \
  --statistics Maximum \
  --period 60 \
  --start-time <start-time> \
  --end-time <end-time> \
  --region ap-south-1
```

Compare the observed lag with the RPO. For a 5-minute RPO, a measured lag of 3 seconds is comfortably inside the target; a lag of 8 minutes is not.

6.3 Check RDS events

```
aws rds describe-events \
  --source-type db-instance \
  --source-identifier jhanvi-cars-dr \
  --duration 60 \
  --region ap-south-1
```

6.4 Check CloudWatch health metrics

```
aws cloudwatch list-metrics \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=jhanvi-cars-dr \
  --region ap-south-1
```

Review ReplicaLag, FreeStorageSpace, CPUUtilization, DatabaseConnections, IOPS and latency. A replica can be technically replicating while storage or resource pressure makes it unsafe to rely on.

7. PostgreSQL-Level Validation

On the primary, inspect replication/WAL information where the environment permits:

```
SELECT
    client_addr,
    state,
    sync_state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn
FROM pg_stat_replication;
```

Useful LSN checks:

```
SELECT pg_current_wal_lsn();
```

```
-- On a PostgreSQL standby:
SELECT pg_last_wal_receive_lsn();
SELECT pg_last_wal_replay_lsn();

SELECT pg_is_in_recovery();

SELECT now() - pg_last_xact_replay_timestamp();
```

Use timestamps as supporting evidence, not as the sole RPO measurement. LSN/WAL position, RDS replication metrics, controlled test transactions and application validation together provide stronger evidence.

8. Controlled Replication Test

For a planned DR exercise, use a dedicated test table rather than modifying production business data merely to prove replication.

```
CREATE TABLE IF NOT EXISTS dr_test (
    id BIGSERIAL PRIMARY KEY,
    test_id TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO dr_test(test_id)
VALUES ('DR-TEST-001');

SELECT *
FROM dr_test
WHERE test_id = 'DR-TEST-001';
```

After the change has had time to replicate, run the same SELECT on the DR database. The record appearing there verifies actual data propagation.

9. Database Consistency Checks

```
SELECT count(*) FROM customers;
SELECT count(*) FROM drivers;
SELECT count(*) FROM vehicles;
SELECT count(*) FROM bookings;

SELECT id, created_at
FROM bookings
ORDER BY created_at DESC
LIMIT 10;
```

Also compare required schemas, extensions, roles, permissions and other application dependencies. Row counts alone do not prove identical data.

10. Determine the Last Known Replicated State

Last known primary transaction means the most recent committed change on the primary that you can establish as part of the database's known state before failure. For DR, distinguish the primary's last transaction from the latest transaction received/replayed on the DR side.

```
Primary:
SELECT pg_current_wal_lsn();

Standby:
SELECT pg_last_wal_receive_lsn();
SELECT pg_last_wal_replay_lsn();
SELECT pg_last_xact_replay_timestamp();
```

Example: if the disaster occurs at 10:30:00 and the latest confirmed replayed state corresponds to approximately 10:29:57, the observed RPO is about 3 seconds. Record the actual evidence used to make that calculation.

11. Phase E – Prepare for Failover

For a controlled DR test, record the disaster start time, stop or redirect application writes if appropriate, and perform a final replication check. The goal is to minimize divergence before promotion.

```
# Example lab-only action:
kubectl -n jhanvi-cars scale deployment jhanvi-cars-backend --replicas=0

# Verify:
kubectl -n jhanvi-cars get pods

# Then perform the final replication/lag checks.
```

12. Phase F – Promote the DR Database

```
aws rds promote-read-replica \
  --db-instance-identifier jhanvi-cars-dr \
  --region ap-south-1
```

Monitor promotion:

```
aws rds describe-db-instances \
  --region ap-south-1 \
  --db-instance-identifier jhanvi-cars-dr \
  --query 'DBInstances[0].DBInstanceStatus'
```

13. Confirm DR Database Is Writable

```
psql -h <dr-endpoint> -U <user> -d <database>

SELECT pg_is_in_recovery();

CREATE TABLE IF NOT EXISTS dr_failover_test (
    id BIGSERIAL PRIMARY KEY,
    message TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

INSERT INTO dr_failover_test(message)
VALUES ('DR database is writable');

SELECT *
FROM dr_failover_test
ORDER BY id DESC
LIMIT 1;
```

Before promotion, a standby normally reports pg_is_in_recovery() = true. After successful promotion to a writable primary, it should report false.

14. Phase G – Move the Application to DR

```
kubectl -n jhanvi-cars get deployment
kubectl -n jhanvi-cars get pods
kubectl -n jhanvi-cars get svc
kubectl -n jhanvi-cars get ingress

kubectl -n jhanvi-cars get deployment jhanvi-cars-backend -o yaml

kubectl -n jhanvi-cars rollout restart deployment jhanvi-cars-backend

kubectl -n jhanvi-cars rollout status deployment jhanvi-cars-backend

kubectl -n jhanvi-cars logs deployment/jhanvi-cars-backend --tail=100
```

Update the database endpoint through your Secret, ConfigMap, AWS Secrets Manager/External Secrets integration or other configuration mechanism. Do not bake a region-specific database endpoint into the container image.

15. Validate DR ALB and Application

```
aws elbv2 describe-target-groups --region ap-south-1

aws elbv2 describe-target-health \
```

```
  --target-group-arn <target-group-arn> \
  --region ap-south-1

curl https://<dr-alb-dns-name>/health
```

Validate healthy ALB targets, Kubernetes readiness, application logs and a database-backed health check. Then perform a real synthetic transaction such as login → retrieve drivers → create booking → retrieve booking.

16. Phase H – Route 53 Failover

```
aws route53 list-resource-record-sets   --hosted-zone-id <zone-id>
```

For a manual failover exercise, update the DNS record so api.jhanvicars.in points to the DR ALB. For an automated design, configure an appropriate Route 53 failover/health-check strategy.

```
aws route53 change-resource-record-sets \
  --hosted-zone-id <zone-id> \
  --change-batch file://dr-failover.json

dig api.jhanvicars.in
nslookup api.jhanvicars.in
curl https://api.jhanvicars.in/health
```

17. Measure RTO and RPO

Metric	Example	How to establish it
RPO target	≤ 5 minutes	Business requirement
Observed RPO	~3 seconds	Last confirmed replicated/replayed state vs failure
RTO target	≤ 30 minutes	Business requirement
Observed RTO	12m 35s	Disaster timestamp to successful service recovery

18. Failback

Do not simply restart the old primary after the DR database has accepted writes. Doing so can create split-brain or divergent data. First recover the old primary environment, establish a safe synchronization path from the current DR primary, wait for data synchronization, validate it, deploy the application, and only then perform a controlled DNS failback.

```
DR PRIMARY
    |
    | synchronize
    v
Recovered PRIMARY
    |
    v
Validate data and application
    |
    v
Switch Route 53 back
    |
    v
PRIMARY ACTIVE
```

19. Snapshot/Restore Alternative

For a cheaper DR lab, use cross-Region snapshots rather than a continuously running replica. This usually gives higher RTO/RPO but reduces always-on database cost.

```
aws rds create-db-snapshot \
  --db-instance-identifier jhanvi-cars-prod \
  --db-snapshot-identifier jhanvi-cars-dr-test \
  --region ap-south-2

aws rds copy-db-snapshot \
```

```
  --source-db-snapshot-identifier <source-snapshot-arn> \
  --target-db-snapshot-identifier jhanvi-cars-dr-test \
  --source-region ap-south-2 \
  --region ap-south-1

aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier jhanvi-cars-dr \
  --db-snapshot-identifier jhanvi-cars-dr-test \
  --region ap-south-1
```

20. Production-Style DR Checklist

Area	Checks
Replication	[ ] Running [ ] Lag within RPO [ ] No errors [ ] WAL/LSN healthy
Database	[ ] Storage healthy [ ] Schema/extensions/roles verified [ ] Critical data checked
Network	[ ] EKS → RDS 5432 works [ ] SG/routes/NACL/DNS validated
Application	[ ] Correct image [ ] Secrets available [ ] Pods ready [ ] DB connection works
ALB	[ ] Target group healthy [ ] Health checks pass [ ] HTTPS works
Failover	[ ] Disaster timestamp recorded [ ] DB promoted [ ] DNS switched
Validation	[ ] Synthetic transaction succeeds [ ] RPO recorded [ ] RTO recorded
Failback	[ ] Primary recovered [ ] Data synchronized [ ] Application validated [ ] DNS switched back

21. Key Commands – Quick Reference

```
# RDS
aws rds describe-db-instances ...
aws rds describe-events ...
aws rds promote-read-replica ...

# CloudWatch
aws cloudwatch get-metric-statistics ...

# PostgreSQL
SELECT pg_is_in_recovery();
SELECT pg_current_wal_lsn();
SELECT pg_last_wal_receive_lsn();
SELECT pg_last_wal_replay_lsn();
SELECT pg_last_xact_replay_timestamp();

# Kubernetes
kubectl get nodes
kubectl -n jhanvi-cars get pods
kubectl -n jhanvi-cars get svc
kubectl -n jhanvi-cars get ingress
kubectl -n jhanvi-cars logs deployment/jhanvi-cars-backend
kubectl -n jhanvi-cars rollout status deployment/jhanvi-cars-backend

# ALB
aws elbv2 describe-target-groups ...
aws elbv2 describe-target-health ...

# DNS
dig api.jhanvicars.in
nslookup api.jhanvicars.in
curl https://api.jhanvicars.in/health
```

22. Senior DevOps Interview Summary

A strong DR answer is not simply 'the replica is healthy'. Validate replication status, replication lag, replication errors, WAL/LSN state, database consistency, connectivity, application health, ALB target health and a synthetic transaction. Record the observed RPO and RTO. Promote only after the validation gates pass, and treat failback as a separate controlled operation.

Safety note: Commands that stop workloads, promote databases, modify DNS or alter production infrastructure are appropriate for a controlled DR lab/runbook only after review and approval in real environments.